ECE 6257-GDL, Fall 2026
Problem Set 3 Solutions

Michael Incorvaia

Contents

1 Problem 1: GANs on MNIST (40 points) 3
- 1.1 Part 1: Libraries and Utility Functions . . . 3
  - 1.1.1 Normalization to $[-1, 1]$ . . . 3
  - 1.1.2 Minimax training and Jensen–Shannon divergence . . . 4
  - 1.1.3 Generator update . . . 4
- 1.2 Part 2: Vanilla GAN . . . 4
  - 1.2.1 Noise sampling . . . 4
  - 1.2.2 Discriminator . . . 4
  - 1.2.3 Generator . . . 5
- 1.3 Part 3: GAN Loss . . . 5
  - 1.3.1 Discriminator . . . 5
  - 1.3.2 Generator . . . 5
- 1.4 Part 4: Training . . . 5
  - 1.4.1 Optimizer . . . 5
  - 1.4.2 Training loop . . . 5
  - 1.4.3 Generated samples . . . 6
- 1.5 Part 5: Least Squares GAN . . . 6
  - 1.5.1 Comparison with the vanilla GAN . . . 6
- 1.6 Part 6: Deep Convolutional GAN . . . 7
  - 1.6.1 Comparison with the fully connected GANs . . . 8
- 1.7 Part 7: Comparison with real MNIST . . . 8
  - 1.7.1 Can the generator win on finite samples? . . . 9

2 Problem 2: EBMs on MNIST (40 points) 10
- 2.1 Part 0/1: Contrastive divergence . . . 10
  - 2.1.1 Effect on the energy of correct and wrong samples . . . 11
  - 2.1.2 When the push and pull balance . . . 11
- 2.2 Part 2: Model construction and training . . . 11
  - 2.2.1 Energy network . . . 11
  - 2.2.2 Langevin sampler . . . 11
  - 2.2.3 Loss . . . 12
  - 2.2.4 Swish vs. ReLU . . . 12
  - 2.2.5 Role of the regularization loss . . . 12
- 2.3 Part 3: Analysis . . . 12
  - 2.3.1 Image generation and number of MCMC steps . . . 13
  - 2.3.2 Out-of-distribution: random noise . . . 14

2.3.3 Out-of-distribution: image transformations 14
2.3.4 Is this a good OOD detector? 15

3 GDL-Bench Challenge Problem (20 points) 17
3.1 (a) Challenge problem 17
3.2 (b) Gold answer and solution 18
3.3 (c) What does the problem test? 20
3.4 (d) Testing AI systems 20

1 Problem 1: GANs on MNIST (40 points)

Question. You will implement a Generative Adversarial Network model and evaluate it under different conditions, using the notebook `gan-mnist.ipynb`.

(a) Part 1: Libraries and utility functions. Why do you think it is convenient to normalize the images to the range $[-1, 1]$ instead of $[0, 1]$? (Bonus) How does the minimax game defined by the objective function of a GAN relate to minimizing the Jensen–Shannon divergence between the training data distribution and the samples obtained from the generator? (Bonus) Why is maximizing the probability of the discriminator making the INCORRECT choice the most practical way to update the generator?

(b) Part 2: Vanilla GAN. Implement the noise sampling function as well as the generator and discriminator networks.

(c) Part 3: GAN Loss. Implement the necessary losses for both networks.

(d) Part 4: Training Function. Implement the optimizer construction, review and run the training loop cells. How do the generated samples look?

(e) Part 5: Least Squares GAN. Implement the Least-Squares loss function and evaluate. How do the resulting generated samples compare to those produced by the Vanilla GAN? Why do you think this is the case?

(f) Part 6: Deeply Convolutional GANs. Implement the generator and discriminator architectures and evaluate the resulting model. How do the generated samples compare to the two previous configurations? Why does adding convolutional layers have this effect?

(g) Part 7: Compare generated images. Evaluate the generated samples from the three previous models against the real MNIST images. Can you distinguish the original from the fake ones in all three cases? Do you think an equilibrium exists where the generator wins, i.e. the discriminator ends up unable to distinguish the two distributions on finite samples? Explain why you believe this would or would not be possible.

Response.

1.1 Part 1: Libraries and Utility Functions

The notebook defines the GAN as the minimax game

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}\left[\log\big(1 - D(G(z))\big)\right], \tag{1}$$

and then replaces the generator step with

$$\max_G \; \mathbb{E}_{z\sim p(z)}[\log D(G(z))]. \tag{2}$$

`preprocess_img` maps real images from $[0, 1]$ to $[-1, 1]$ by $x \mapsto 2x - 1$, and `deprocess_img` maps them back for display.

1.1.1 Normalization to $[-1, 1]$

Scaling to $[-1, 1]$ matches the generator's final tanh, whose outputs lie in $(-1, 1)$. Real and generated images then share one scale, so the discriminator compares like with like and every real pixel value is one the generator can actually produce. The interval is also symmetric about zero, which keeps the inputs to both networks roughly zero-mean, and it matches the range of the latent noise.

1.1.2 Minimax training and Jensen–Shannon divergence

For a fixed generator, the discriminator that maximizes (1) is

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}. \tag{3}$$

Substituting D_G^* back in gives

$$\begin{aligned} C(G) &= -\log 4 + \mathrm{KL}\left(p_{\text{data}} \middle\| \frac{p_{\text{data}} + p_g}{2}\right) + \mathrm{KL}\left(p_g \middle\| \frac{p_{\text{data}} + p_g}{2}\right) \\ &= -\log 4 + 2\,\mathrm{JSD}(p_{\text{data}} \| p_g). \end{aligned} \tag{4}$$

The inner maximization turns the game into a constant plus twice the Jensen–Shannon divergence between the data and generator distributions, so minimizing over G minimizes that divergence. The minimum, $-\log 4$, is reached only when $p_g = p_{\text{data}}$, where $D_G^*(x) = 1/2$ everywhere.

1.1.3 Generator update

Early in training the generator is weak and $D(G(z)) \approx 0$. With $D = \sigma(s)$ for a logit s, the minimax generator term $\log(1 - D(G(z)))$ has gradient $-D(G(z))$ with respect to s, which goes to zero there, so the generator gets almost no signal.

The non-saturating update (2) instead minimizes $-\mathbb{E}_z[\log D(G(z))]$, whose gradient with respect to s is $-(1 - D(G(z)))$. That is largest exactly when $D(G(z))$ is near 0. It maximizes the probability that the discriminator is wrong on generated data, keeps the same fixed point ($p_g = p_{\text{data}}$, $D = 1/2$), and gives much larger gradients early on. The discriminator still trains on (1).

1.2 Part 2: Vanilla GAN

1.2.1 Noise sampling

`sample_noise(batch_size, dim)` draws `torch.rand` of shape (`batch_size`, `dim`), uniform on $[0, 1]$, and maps $u \mapsto 2u - 1$. The result is the uniform prior on $[-1, 1]$.

1.2.2 Discriminator

A fully connected classifier with biases and LeakyReLU slope 0.01:

Layer	Shape	Activation
Flatten	image $\to$ 784	none
Linear	$784 \to 256$	LeakyReLU(0.01)
Linear	$256 \to 256$	LeakyReLU(0.01)
Linear	$256 \to 1$	none

It outputs one logit per image, shape [`batch_size`, 1]. There is no sigmoid, so the loss can use a numerically stable cross-entropy on logits. The network has 267,009 parameters.

1.2.3 Generator

Layer	Shape	Activation
Linear	`noise_dim` $\to 1024$	ReLU
Linear	$1024 \to 1024$	ReLU
Linear	$1024 \to 784$	tanh

The final tanh puts every pixel in $(-1, 1)$, the range of the preprocessed real images. With `noise_dim` $= 4$, as in the notebook's parameter check, the network has 1,858,320 parameters; training uses `NOISE_DIM` $= 96$.

1.3 Part 3: GAN Loss

Both losses are minibatch means of the provided stable binary cross-entropy on logits,

$$bce(s, y) = -y \log \sigma(s) - (1 - y) \log\big(1 - \sigma(s)\big),$$

with real targets 1 and fake targets 0. Scores of shape $[N, 1]$ are flattened to length N so they line up with the labels, which are cast to the scores' device with `.type(dtype)`.

1.3.1 Discriminator

$$\ell_D = -\mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] - \mathbb{E}_{z \sim p(z)}\big[\log\big(1 - D(G(z))\big)\big] .$$

In code this is `loss_real + loss_fake`: real logits against ones plus fake logits against zeros. Since `bce_loss` returns a mean, the sum matches the two expectations. Minimizing it raises $D(x)$ on real images and lowers $D(G(z))$ on fakes.

1.3.2 Generator

The non-saturating objective from Part 1, written for minimization:

$$\ell_G = -\mathbb{E}_{z \sim p(z)}[\log D(G(z))] .$$

It is the cross-entropy between fake logits and ones, which pushes $D(G(z))$ toward 1.

1.4 Part 4: Training

1.4.1 Optimizer

`get_optimizer` returns Adam with learning rate 10^{-3} and betas $(0.5, 0.999)$, used for every network in the notebook. $\beta_1 = 0.5$ is the usual GAN choice: a shorter gradient memory suits a discriminator that keeps moving.

1.4.2 Training loop

The loop is provided. Each epoch runs the 390 full minibatches of 128 from the 50,000 training images (the incomplete last batch is skipped), for 10 epochs. Each iteration takes one discriminator step, then one generator step. Real images are scaled by $2(x - 1/2)$, the same map as `preprocess_img`. The discriminator step detaches the generated images, so only D is updated. The generator step draws fresh noise and scores the fakes with the updated D; the gradients this leaves on D are cleared by the next iteration's `zero_grad`. Every 250 iterations a fixed noise batch, drawn once at the start, is passed through G and saved to `v_gan/image_*.png`, so the same latent codes can be followed through training.

1.4.3 Generated samples

At iteration 0 every tile is gray pixel noise. By iteration 250 the mass has collapsed into bright, fuzzy blobs on a still-gray background, most with the same rough 8-like shape. By iteration 1000 the backgrounds are dark and the blobs have become soft digits: nines, sevens, zeros, and ones are recognizable, with speckled edges. At iterations 3000 and 3750 about half of the fixed seeds are clearly a digit (1, 4, 6, 7); the rest are broken or noisy, and speckle remains around most strokes. The discriminator loss stays near 1.0–1.4, around the equilibrium value $2\log 2 \approx 1.39$, and the generator loss settles near 0.8–1.1 after an early spike. The samples cover a spread of digit classes rather than a single mode.

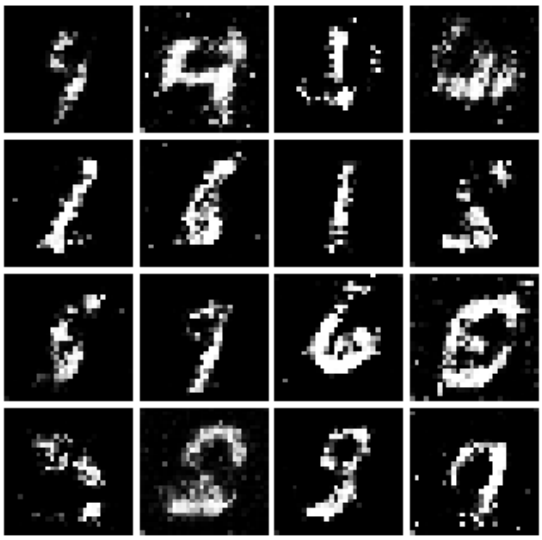


Figure 1: Vanilla GAN, this run's fixed latent codes at iteration 3750.

1.5 Part 5: Least Squares GAN

The networks and training loop stay the same; only the losses change, to equation (9) of Mao et al.:

$$\ell_D = \tfrac{1}{2}\mathbb{E}_{x\sim p_{\text{data}}}\Big[\big(D(x)-1\big)^2\Big] + \tfrac{1}{2}\mathbb{E}_{z\sim p(z)}\Big[\big(D(G(z))\big)^2\Big], \tag{5}$$

$$\ell_G = \tfrac{1}{2}\mathbb{E}_{z\sim p(z)}\Big[\big(D(G(z))-1\big)^2\Big]. \tag{6}$$

Each expectation is a minibatch mean of the raw scores. Against `gan-checks.npz` the errors are 1.53×10^{-8} for the discriminator and 2.78×10^{-9} for the generator, both under the 10^{-7} tolerance.

1.5.1 Comparison with the vanilla GAN

At iteration 1000 the LS samples are still clusters of salt-and-pepper pixels, behind the vanilla grid at the same point. By iterations 3000 and 3750 most fixed seeds are a recognizable digit (5, 3, 9, 1, 4, 6, 7), with thicker and more complete strokes than the vanilla samples, where only about half

are clearly digits. The gain is modest in this run: the speckle around the strokes is about as heavy as in the vanilla grid, a few seeds are still blobs, and the 4 repeats several times.

The more complete strokes come from the loss. The vanilla generator's cross-entropy saturates once a fake is on the real side of the decision boundary, so there is little gradient left to fix speckle or broken strokes. The LS generator loss has gradient proportional to $D(G(z)) - 1$, which stays large until the score actually reaches 1, so fakes that already fool the classifier are still pulled toward the real data. With labels satisfying $b - c = 1$ and $b - a = 2$, Mao et al. show that the LS objective minimizes a Pearson χ^2 divergence. The 0/1 labels used here do not meet that condition, but the gradient argument still holds.

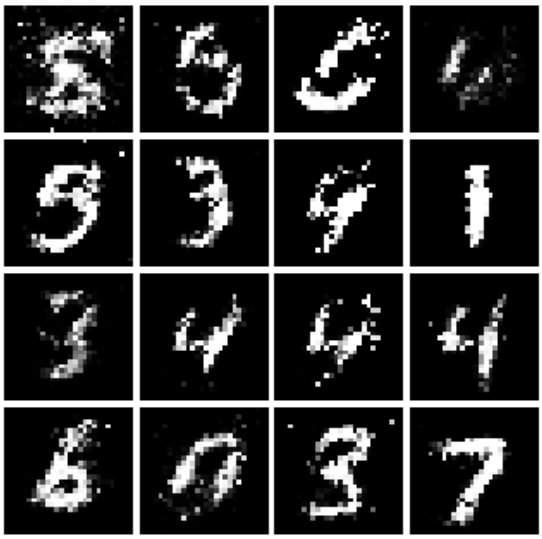


Figure 2: LSGAN, this run's fixed latent codes at iteration 3750.

1.6 Part 6: Deep Convolutional GAN

The loss is back to binary cross-entropy; the networks change. The discriminator unflattens to $1 \times 28 \times 28$, then applies conv $1 \to 32$ (5×5), LeakyReLU(0.01), 2×2 max-pool, conv $32 \to 64$ (5×5), LeakyReLU, max-pool, flatten, linear $1024 \to 1024$, LeakyReLU, and linear $1024 \to 1$. It has 1,102,721 parameters and outputs shape $[128, 1]$.

The generator follows the InfoGAN layout: linear to 1024, ReLU, batch norm; linear to $128 \cdot 7 \cdot 7$, ReLU, batch norm; reshape to $128 \times 7 \times 7$; then two transposed convolutions (4×4, stride 2, padding 1) up to $64 \times 14 \times 14$ and $1 \times 28 \times 28$, with ReLU and batch norm between them and tanh at the end. A final flatten returns a 784-vector for the existing training loop. With `noise_dim` $= 4$ it has 6,580,801 parameters. Xavier initialization is applied to every linear and transposed-convolution weight in both networks before training.

1.6.1 Comparison with the fully connected GANs

Iteration 0 is noise. By iteration 1000 the fixed seeds are already clean digits on a black background, ahead of where either fully connected model was at iteration 3750. At iteration 3750 the strokes are smooth and the backgrounds clean: $1, 3, 7, 9, 8, 5, 8, 3, 1, 1, 5, 0, 9, 6, 7, 3$. A few tiles are slightly soft and the 6 is a bit malformed, but there is almost none of the salt-and-pepper noise in the vanilla and LS grids.

Convolution is the reason. A fully connected generator treats the 784 pixels as one vector, so nothing in its wiring says strokes are local or that the same edge can appear anywhere; the samples show that as speckle and broken contours. A convolutional discriminator looks at local patches with shared filters, so it judges stroke shape rather than a bag of pixels. The transposed convolutions build the image by upsampling a 7×7 spatial code, so neighboring pixels are produced together, and batch norm keeps the deeper generator stable. The result is coherent strokes much earlier in training.

The losses are less flattering than the images. Over the last 1000 iterations the discriminator loss sits near 1.0–1.1 and the generator loss near 0.9–1.3, on either side of the chance-level pair $2 \log 2 \approx 1.39$ and $\log 2 \approx 0.69$, so the discriminator still separates real from fake better than chance.

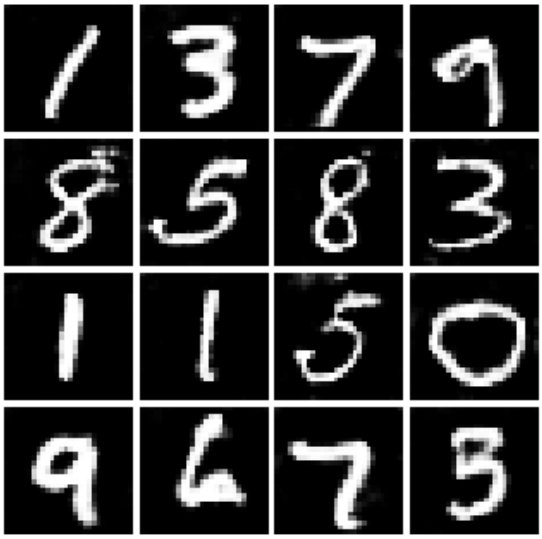


Figure 3: DCGAN, this run's fixed latent codes at iteration 3750.

1.7 Part 7: Comparison with real MNIST

Stacked against real digits, the three generators separate cleanly. Vanilla samples are easy to pick out: broken strokes, speckled backgrounds, and only about half are sharp digits. LS samples are only slightly harder; most are recognizable, but the strokes stay gritty and speckled. DCGAN samples are the close call, and many would pass as MNIST on their own. In a grid of 50 fakes over 50 reals (Figure 4), the real half is slightly crisper and the fake half still has the occasional stray pixel, soft edge, or slightly wrong stroke. That is enough to tell the two blocks apart, but not to label every single DCGAN image with confidence.

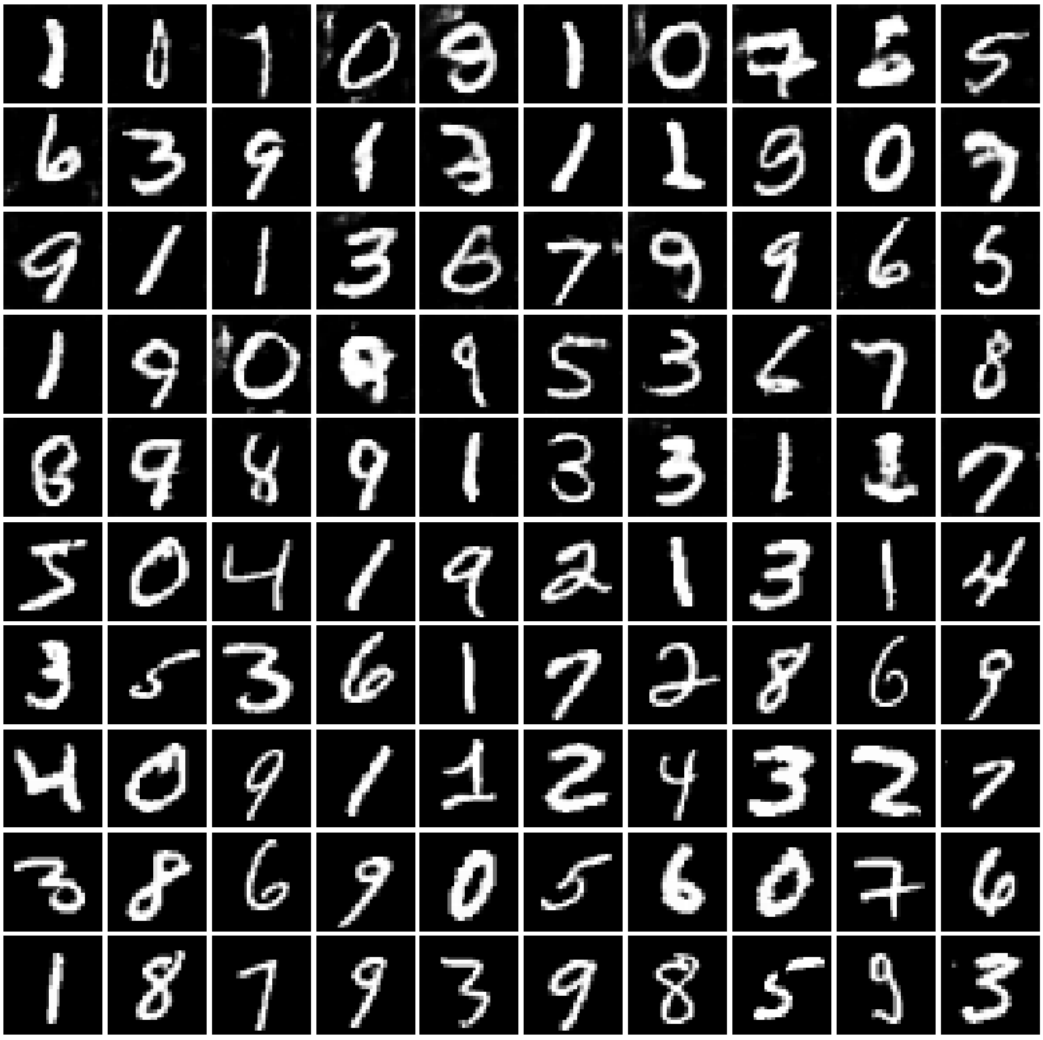


Figure 4: DCGAN samples (top five rows) above real MNIST digits (bottom five rows).

1.7.1 Can the generator win on finite samples?

The game has an equilibrium, and it is a tie. Goodfellow et al. show that the optimal discriminator is $D = 1/2$ everywhere exactly when $p_g = p_{\text{data}}$, with game value $-\log 4$. The generator never "wins" by making the discriminator wrong; it can only match the data distribution, after which the discriminator's best strategy is to guess. This run does not reach that point even in the loss: the late DCGAN discriminator loss is about 1.0–1.1, not 1.39, so the discriminator being trained still separates real from fake on its minibatches.

Reaching chance in the training loss would still not make the two finite samples indistinguishable. A fresh, high-capacity classifier trained to convergence against a frozen generator can exploit any residual difference: speckle, a broken stroke, a small shift in stroke thickness. Making the two empirical distributions identical would require the generator to reproduce the training set, which none of these runs does. Vanilla and LS samples are obvious fakes, and DCGAN samples, though close, are still separable both by the training discriminator and by eye. So the equilibrium exists in theory, but a generator that makes finite samples truly indistinguishable is not something this run reaches, and it is not what one-step alternating updates on finite networks are set up to guarantee.

2 Problem 2: EBMs on MNIST (40 points)

Question. You will implement an Energy Based Model and evaluate it for generative performance and anomaly detection capacity, using the notebook `ebm-mnist.ipynb`.

(a) Part 0/1: Libraries and utility functions. How does the contrastive divergence objective change the energy of the correct/wrong samples? What do you think would need to happen for the push/pull forces on the diagram to balance out?

(b) Part 2: Model construction and training. Implement the model construction, MCMC sampling steps, and loss calculation, and run the implemented model.

- Why do you think that the Swish activation function is more convenient than the standard ReLU for this model and task? *Hint: visualize both activation functions and think about how they would affect the backward propagation of the gradient through the network.*
- What is the role of the regularization loss? How do you think the resulting data would look if we were not using it?

(c) Part 3: Analysis. Sample a few images using the MCMC method. How many MCMC steps are required to sample reasonable images from the trained model? Is this number different for different digits? Then complete the transformation cells to test the out-of-distribution performance of the model.

- For pure random noise inputs: are the scores in line with what would be expected for a good anomaly detection method? Why do you think the score for the 'true' images is so close to zero?
- For the image transformations: do the scores of the transformed images differ significantly from the original ones? Do you think this model would make a good out-of-distribution detector? Provide a detailed explanation of why this would or would not be the case.

Response.

The network outputs a scalar score $f_\theta(x)$, and the notebook defines the model density as

$$q_\theta(x) = \frac{\exp\big(f_\theta(x)\big)}{Z_\theta}, \qquad f_\theta(x) = -E_\theta(x), \tag{7}$$

so a high score means low energy and high unnormalized probability. The normalizer Z_θ integrates $\exp(f_\theta)$ over all images and is intractable. Images are scaled to $[-1, 1]$, which is also the box the sampler is clipped to.

2.1 Part 0/1: Contrastive divergence

Maximum likelihood minimizes $\mathcal{L}(\theta) = -\mathbb{E}_{p(x)}[\log q_\theta(x)]$. Since $\log q_\theta(x) = f_\theta(x) - \log Z_\theta$ and $\nabla_\theta \log Z_\theta = \mathbb{E}_{q_\theta(x)}[\nabla_\theta f_\theta(x)]$,

$$\nabla_\theta \mathcal{L} = \mathbb{E}_{q_\theta(x)}\big[\nabla_\theta f_\theta(x)\big] - \mathbb{E}_{p(x)}\big[\nabla_\theta f_\theta(x)\big]. \tag{8}$$

A descent step therefore raises f_θ on data and lowers it on model samples. In energy terms, that is the pair of forces in the diagram.

2.1.1 Effect on the energy of correct and wrong samples

The "correct answer" is a training image $x_{\text{train}} \sim p(x)$; the update lowers its energy, pulling it down the energy curve and making it more probable. The "wrong answer" is a model sample $x_{\text{sample}} \sim q_\theta(x)$; the update raises its energy and makes it less probable. Replacing the model expectation with one sample gives the contrastive divergence loss used later,

$$\ell_{\text{CD}} = f_\theta(x_{\text{sample}}) - f_\theta(x_{\text{train}}) = E_\theta(x_{\text{train}}) - E_\theta(x_{\text{sample}}), \tag{9}$$

and minimizing it is the same push and pull: down on data, up on samples.

2.1.2 When the push and pull balance

The gradient (8) vanishes when $\mathbb{E}_p[\nabla_\theta f_\theta] = \mathbb{E}_{q_\theta}[\nabla_\theta f_\theta]$, which holds once $q_\theta = p$. Model samples then land on the same images as the training data, so the upward push on "wrong" samples and the downward pull on "correct" ones act on the same regions and cancel. Until then, mass sitting off the data keeps being pushed up and real digits keep being pulled down.

2.2 Part 2: Model construction and training

2.2.1 Energy network

f_θ is a small CNN with Swish activations, $x\,\sigma(x)$. On a $1 \times 28 \times 28$ image:

Layer	Shape	Activation
Conv 5×5, stride 2, pad 4	$1 \to 16$, 16×16	Swish
Conv 3×3, stride 2, pad 1	$16 \to 32$, 8×8	Swish
Conv 3×3, stride 2, pad 1	$32 \to 64$, 4×4	Swish
Conv 3×3, stride 2, pad 1	$64 \to 64$, 2×2	Swish
Flatten	256	none
Linear	$256 \to 64$	Swish
Linear	$64 \to 1$	none

The output is squeezed to one score $f_\theta(x) = -E_\theta(x)$ per image. The network has 76,993 parameters.

2.2.2 Langevin sampler

Samples come from short-run Langevin dynamics in the box $[-1, 1]$. Each of K steps, with step size ν, is

$$\begin{aligned} x &\leftarrow \text{clip}\big(x + \omega,\, -1,\, 1\big), && \omega \sim \mathcal{N}(0,\, 0.005^2), \\ g &\leftarrow \text{clip}\big(\nabla_x E_\theta(x),\, -0.03,\, 0.03\big), \\ x &\leftarrow \text{clip}\big(x - \nu g,\, -1,\, 1\big). \end{aligned} \tag{10}$$

The gradient of $E_\theta = -f_\theta$ is taken with the weights frozen and clipped so a single step cannot jump across the image; $x - \nu g$ walks downhill in energy. Training uses $K = 60$ and $\nu = 10$. The generation plots use $K = 256$ from fresh noise.

A replay buffer of up to 8,192 images supplies the starting points: 95% of each batch comes from the buffer and 5% from fresh uniform noise, and the new samples go to the front of the buffer. These warm starts are why 60 steps suffice during training, since most chains already start near a mode.

2.2.3 Loss

Real images get pixel noise with standard deviation 0.005 and are clipped back to $[-1, 1]$. The network scores the real and sampled batches together, and with f_{real}, f_{fake} those scores,

$$\ell_{\text{CD}} = \mathbb{E}[f_{\text{fake}}] - \mathbb{E}[f_{\text{real}}], \tag{11}$$

$$\ell_{\text{reg}} = \mathbb{E}\big[f_{\text{real}}^2 + f_{\text{fake}}^2\big], \tag{12}$$

$$\ell = \ell_{\text{CD}} + \alpha\, \ell_{\text{reg}}, \qquad \alpha = 0.1. \tag{13}$$

ℓ_{CD} is the batch version of (9). The optimizer is Adam with learning rate 10^{-4} and betas $(0, 0.999)$; first-moment momentum is off because the loss surface moves as the negative samples change. The learning rate decays by 0.97 per epoch, and gradients are clipped to global norm 0.1. The model was trained for the full 60 epochs with batch size 128 on a single NVIDIA H100 on PACE ICE. The checkpoint used below is the best one by validation contrastive divergence, from epoch 58 (counting from 0, so the 59th of 60) at step 27,612.

2.2.4 Swish vs. ReLU

Swish is smooth: for positive inputs it approaches $y = x$, and for negative inputs it dips slightly below zero and returns toward it. Its derivative,

$$\sigma(x)\big(1 + x\big(1 - \sigma(x)\big)\big),$$

is continuous and is zero only at the single point $x \approx -1.28$. ReLU's derivative is exactly zero on all of $x < 0$ and jumps at the origin.

That matters because sampling follows $\nabla_x f_\theta(x)$, which has to backpropagate through every activation. Wherever a ReLU unit is off, that part of the energy surface is flat, the input gradient dies, and the Langevin chain stalls. Swish still passes a slope through those regions, so both the sampler and the parameter updates keep a direction. ReLU's kink also makes $\nabla_x f_\theta$ discontinuous, which makes Langevin steps jumpy; Swish keeps f_θ smooth.

2.2.5 Role of the regularization loss

Contrastive divergence only sees the gap $f_{\text{fake}} - f_{\text{real}}$. That gap can grow without bound by sending real scores to $+\infty$ and fake scores to $-\infty$, which doesn't change which images are relatively likely but turns the energy surface into a cliff. Langevin steps would then always hit the ± 0.03 gradient cap and slam into the ± 1 walls, the samples would become saturated noise or harsh blobs, and training would diverge.

The penalty $\alpha\, \mathbb{E}[f_{\text{real}}^2 + f_{\text{fake}}^2]$ keeps both scores near zero, so training has to shape the surface rather than just stretch it. It is also why real-image scores sit near zero in Part 3: zero is the scale the regularizer picked, not a statement that the probability is 1.

2.3 Part 3: Analysis

Everything below uses that checkpoint. For the sampling figure, eight chains start from independent uniform noise and run 256 steps of (10) with $\nu = 10$.

2.3.1 Image generation and number of MCMC steps



Figure 5: Eight Langevin chains started from uniform noise. Columns are the image after the listed number of steps.

Step 1 is noise. By step 8 every chain has a dark background and a bright stroke pattern, and by steps 32–48 most shapes have locked in: the top chain is a 2, the third an L-shaped stroke, the fifth a 7 with a closed top, and the bottom chain a closed loop. Shapes that have to open or add a stroke take longer. The second chain is a closed V until it opens into a hooked 7 around step 128, and the fourth is a C until about step 192, when it grows an inner stroke and settles toward a 6. The sixth and seventh chains are stable by step 64 but never become clear digits. Apart from the fourth chain, little changes between steps 128 and 256 beyond cleaner edges.

So a stable shape takes about 32–64 steps from pure noise, and a recognizable digit, when the chain reaches one, about 64–192. The number depends on the digit: simple strokes and closed loops lock in early, while shapes that need to open or add a stroke keep changing. Only about half of the eight chains end as clear digits, so the model's samples are rougher than its scores on real data suggest. The 60 steps used in training are not comparable, because 95% of those chains start from the replay buffer; from scratch, 60 steps only reach the easy shapes.

2.3.2 Out-of-distribution: random noise

The score is f_θ, and lower means less probable. On 256 images of each kind:

Input	Mean f_θ	Spread
Uniform noise on $[-1, 1]$	-15.70	all scores in $[-16.11, -15.33]$
MNIST test digits	0.01	std 0.02, range $[-0.09, 0.04]$
MNIST training digits	0.00	std 0.03

A gap of about 16 against a real-digit spread of 0.02 is exactly what a good anomaly detector should show for unstructured noise.

Real digits sit near zero because of the regularizer in (13). The contrastive term only needs $f(\text{real})$ above $f(\text{fake})$, and the quadratic penalty is smallest when both are near zero, so the optimizer parks data at about 0 and pushes everything else far negative. The score is an unnormalized log-density, so zero just means "on the scale ℓ_{reg} chose."

2.3.3 Out-of-distribution: image transformations

The same 256 test digits were corrupted three ways: Gaussian noise of scale 0.3, clipped to $[-1, 1]$; a flip of both spatial axes; and the digit shrunk into the bottom-right corner on a background of -1.

Transform of the 256 test digits	Mean f_θ
Original	0.01
Noise, scale 0.3	-0.17
Flipped vertically and horizontally	-0.01
Shrunk into the corner	-0.05

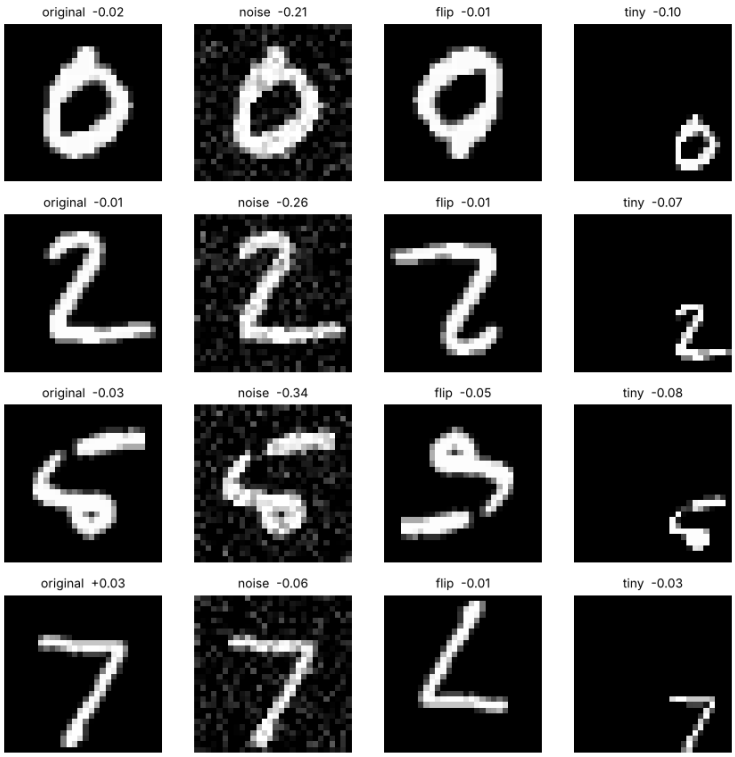


Figure 6: Scores f_θ for four test digits after the three transformations. Titles are the score of that image.

Noise moves the score by only a few tenths. In Figure 6 the noisy 0 goes from -0.02 to -0.21, the noisy 5 from -0.03 to -0.34, and the noisy 7 only from $+0.03$ to -0.06, still inside the range of real scores (the worst test digit in the batch scored -0.09). Flipped digits are no longer valid instances of their class, yet the batch mean moves by only 0.02, and the four flips in the figure by at most 0.04. The shrunk digits are the wrong size in the wrong place and move only slightly more: the shrunk 0 goes from -0.02 to -0.10, just below the worst real digit, and the shrunk 7 from $+0.03$ to -0.03.

2.3.4 Is this a good OOD detector?

It is a good detector of the kind of negatives it was trained against, and a weak detector of anything else.

Unstructured noise sits about 16 units below the data with no overlap, so any threshold between -1 and -15 rejects it and keeps ordinary digits. That is what contrastive divergence with short-run Langevin is built for: the negative phase is noise or a short chain that hasn't reached the data yet, so the model learns to score that region low.

Flips and large changes of scale and position are also outside the MNIST generative process, but the scores barely register them. The gap from a real digit to a flipped or shrunk one is a few hundredths, comparable to the spread of real digits (std 0.02), so a threshold that rejected them would also reject ordinary test digits. Pixel noise shifts the mean to -0.17, a real effect, but individual noisy digits still overlap the worst real scores, so the sets are not separable.

The cause is specific. The negative samples never include flipped or rescaled digits, so nothing in (8) raises the energy of an image that still contains plausible local strokes, and the CNN mostly

scores those strokes; a flipped 7 and a tiny 2 both have them. The regularizer then squeezes every in-distribution score toward 0, leaving almost no room for a slightly wrong image to stand apart. The model responds to "stroke on background at roughly the right contrast" and is insensitive to class identity, orientation, and layout. That is enough to reject pure noise, but not enough for a general out-of-distribution detector.

3 GDL-Bench Challenge Problem (20 points)

Question. As part of each homework, you will contribute one candidate problem to GDL-Bench, a benchmark for evaluating the reasoning capabilities of current AI systems in generative and geometric deep learning. For this homework, design one original challenge problem related to one or more of the topics covered in this assignment. The problem should require genuine mathematical, conceptual, algorithmic, or implementation-level reasoning, and should not be answerable by simply recalling a definition or looking up a fact.

(a) **Challenge problem.** Write the exact problem as you would like it to appear in GDL-Bench. It must be original, self-contained, technically well-defined, and unambiguous.

(b) **Gold answer and solution.** Provide the correct final answer together with a concise but complete solution, enough for another student or instructor to independently verify the answer. Whenever possible, design the problem so that the final answer can be objectively checked.

(c) **What does the problem test?** Topic; problem type (derivation, conceptual reasoning, counterexample, numerical, coding/debugging, algorithm design, or other); and the core insight in 1–3 sentences, including why a solver with only superficial knowledge might struggle.

(d) **Test AI systems.** Only after the problem and solution are finalized, give the exact same problem to at least two current AI models, with no extra hints and without the solution. For each model report the name/version, whether the answer was Correct, Partially Correct, or Incorrect, and a 1–3 sentence explanation of any important error. A problem does not need to defeat the models to receive full credit.

Quality bar: original, relevant, self-contained, unambiguous, verifiable, and reasoning-intensive. Grading: relevance and originality (4); technical correctness (6); clarity (4); depth of reasoning (4); complete metadata and AI evaluation (2).

Response.

The full, agent-runnable version of this problem (task statement, reference solution, and automated tests) is in `gdlbench_submission/`. The statement below is the text of `task/TASK.md`.

3.1 (a) Challenge problem

Which generator does each GAN objective prefer?

All logarithms are natural, and all distributions live on the nonnegative integers. The data distribution p is uniform on $\{0, 1, 2, 3\}$. For each positive integer k, the generator distribution q_k is uniform on $\{0, 1, \ldots, k-1\}$. Conventions: $\log 0 = -\infty$, $0 \cdot (\pm\infty) = 0$, and an expectation that puts positive probability on an integrand equal to $+\infty$ is $+\infty$.

Definitions.

- $\mathrm{JSD}(p, q) = \frac{1}{2} D_{\mathrm{KL}}(p \| m) + \frac{1}{2} D_{\mathrm{KL}}(q \| m)$, with $m = (p + q)/2$.
- *Minimax GAN.* For a discriminator D mapping integers to $[0, 1]$,

$$V(D, q) = \mathbb{E}_{x \sim p}[\log D(x)] + \mathbb{E}_{x \sim q}[\log(1 - D(x))].$$

Let D_q^* maximize $V(D, q)$ (its values off the support of $p + q$ do not matter), and define

$$V_\star(q) = V(D_q^*, q), \qquad L_{\mathrm{MM}}(q) = \mathbb{E}_{x \sim q}[\log(1 - D_q^*(x))], \qquad L_{\mathrm{NS}}(q) = \mathbb{E}_{x \sim q}[-\log D_q^*(x)].$$

- *Least-squares GAN* with labels $a = -1$ (generated), $b = 1$ (real), and generator target $c = 0$. The discriminator D_q^{LS} minimizes $\frac{1}{2}\mathbb{E}_p[(D-1)^2] + \frac{1}{2}\mathbb{E}_q[(D+1)^2]$ over real-valued D, and the generator criterion is

$$V_{\mathrm{LS}}(q) = \tfrac{1}{2}\mathbb{E}_{x\sim p}\big[(D_q^{\mathrm{LS}}(x)-0)^2\big] + \tfrac{1}{2}\mathbb{E}_{x\sim q}\big[(D_q^{\mathrm{LS}}(x)-0)^2\big].$$

The generator wants each of $V_\star$, L_{MM}, L_{NS}, and V_{LS} to be as small as possible.

Report:

1. `jsd_tie_pairs`: every unordered pair $\{j,k\}$ of distinct positive integers with $\mathrm{JSD}(p,q_j) = \mathrm{JSD}(p,q_k)$ exactly.
2. `ls_tie_pairs`: every unordered pair $\{j,k\}$ of distinct positive integers with $V_{\mathrm{LS}}(q_j) = V_{\mathrm{LS}}(q_k)$ exactly.
3. `ls_value_q2`: the value of $V_{\mathrm{LS}}(q_2)$.
4. `k_hat`: the integer $\hat{k} \neq 2$ such that $\{2,\hat{k}\}$ is in `jsd_tie_pairs`, or `null` if there is none.
5. If $\hat{k}$ exists: the values of $V_\star$, L_{MM}, and L_{NS} at q_2 and at $q_{\hat{k}}$.
6. If $\hat{k}$ exists: for each of $V_\star$, L_{MM}, L_{NS}, which of q_2 and $q_{\hat{k}}$ the generator prefers (lower value), or `"tie"`.

The solver also writes a short derivation that justifies why the lists in items 1 and 2 are *complete*. Answers go in `task/results/answer.json`, with $+\infty$ written as the string `"inf"` and floats to at least 10 significant figures. Pair lists, $\hat{k}$, `"inf"`, and preferences must match exactly; floats must be within 10^{-8}.

3.2 (b) Gold answer and solution

Key	Gold answer	Value
`jsd_tie_pairs`	$\{1,16\},\ \{2,8\}$	
`ls_tie_pairs`	$\{1,16\},\ \{2,8\}$	
`ls_value_q2`	$1/3$	0.333333
`k_hat`	8	
$V_\star(q_2) = V_\star(q_8)$	$\log 2 - \frac{3}{2}\log 3$	-0.954771
$L_{\mathrm{MM}}(q_2)$	$\log(2/3)$	-0.405465
$L_{\mathrm{MM}}(q_8)$	$-\frac{1}{2}\log 3$	-0.549306
$L_{\mathrm{NS}}(q_2)$	$\log 3$	1.098612
$L_{\mathrm{NS}}(q_8)$	$+\infty$	
`prefers`	$V_\star$: tie; L_{MM}: q_8; L_{NS}: q_2	

Both criteria depend only on a support ratio. The distributions p and q_k are nested uniforms. Let the smaller one have support size n and the larger one N, and let $r = N/n = \max(k,4)/\min(k,4) \geq 1$. On the shared support $m = (r+1)/(2N)$; off it, $m = 1/(2N)$. Then

$$D_{\mathrm{KL}}(\mathrm{small}\|m) = \log\frac{2r}{r+1}, \qquad D_{\mathrm{KL}}(\mathrm{large}\|m) = \frac{1}{r}\log\frac{2}{r+1} + \Big(1-\frac{1}{r}\Big)\log 2,$$

so

$$\mathrm{JSD}(r) = \log 2 + \tfrac{1}{2}\log r - \tfrac{1}{2}\Big(1 + \tfrac{1}{r}\Big)\log(1+r), \qquad \frac{d\,\mathrm{JSD}}{dr} = \frac{\log(1+r)}{2r^2} > 0.$$

JSD is symmetric, so it does not matter which of p and q_k is the larger one. JSD is therefore a strictly increasing function of r alone, and $\mathrm{JSD}(p, q_j) = \mathrm{JSD}(p, q_k)$ holds iff $r_j = r_k$. For $j \neq k$ this needs one index below 4 and one above it, with $4/j = k/4$, i.e. $jk = 16$. The only integer solutions are $\{1, 16\}$ and $\{2, 8\}$ ($j = 3$ gives $16/3$). The argument covers every positive integer, so the list is complete, and $\hat{k} = 8$.

LSGAN. Minimizing $\frac{1}{2}p(D-1)^2 + \frac{1}{2}q(D+1)^2$ pointwise gives $D^{\mathrm{LS}} = (p-q)/(p+q)$. Substituting,

$$V_{\mathrm{LS}} = \frac{1}{2}\sum_x \frac{(p-q)^2}{p+q} = \frac{1}{2}\Big[\frac{(r-1)^2}{r(r+1)} + \frac{r-1}{r}\Big] = \frac{r-1}{r+1},$$

which is also strictly increasing in r alone. The tie pairs are the same, and $V_{\mathrm{LS}}(q_2) = 1/3$.

Why the ties happen. An f-divergence depends only on the distribution of the likelihood ratio p/q. The pair (p, q_2) has the same ratio structure as (q_8, p): a uniform against a uniform on half its support. Hence $D_f(p\|q_2) = D_g(p\|q_8)$ with the reversed generator $g(u) = u\,f(1/u)$. Every symmetric f-divergence (JSD, the LSGAN χ^2-type term) must tie on this pair. Only asymmetric ones can tell mode dropping (q_2) apart from over-spreading (q_8).

GAN terms. $D^* = p/(p+q)$.

- q_2: $D^* = 1/3$ on $\{0, 1\}$, $D^* = 1$ on $\{2, 3\}$. Then $V_\star = \frac{1}{2}\log\frac{1}{3} + \log\frac{2}{3} = \log 2 - \frac{3}{2}\log 3$, $L_{\mathrm{MM}} = \log\frac{2}{3}$, and $L_{\mathrm{NS}} = \log 3$.
- q_8: $D^* = 2/3$ on $\{0, \ldots, 3\}$, $D^* = 0$ on $\{4, \ldots, 7\}$. Then $V_\star = \log\frac{2}{3} + \frac{1}{2}\log\frac{1}{3} + \frac{1}{2}\log 1 = \log 2 - \frac{3}{2}\log 3$, $L_{\mathrm{MM}} = \frac{1}{2}\log\frac{1}{3} = -\frac{1}{2}\log 3$, and $L_{\mathrm{NS}} = \frac{1}{2}\log\frac{3}{2} + \frac{1}{2}(+\infty) = +\infty$.

$V_\star = 2\,\mathrm{JSD} - \log 4$ ties, as it must. The generator's own minimax term does *not* tie: the $\mathbb{E}_p[\log D^*]$ term also depends on q through D^*, and dropping it leaves a quantity that favors q_8. The non-saturating loss is infinite for any generator that puts mass where the data has none, so it favors q_2, the generator that drops modes. The same game value gives three different verdicts.

Verification. `gt_code/reference_solution.py` evaluates every quantity directly from the distributions (exact fractions wherever no logarithm is involved), scans $k = 1, \ldots, 512$ for ties, and asserts agreement with the closed forms and with $V_\star = 2\,\mathrm{JSD} - \log 4$. `tests/test_solution.py` passes 8/8 on the gold answer, and each of the following mistakes fails at least one test:

Mistake	Fails
Report only $\{2, 8\}$ (miss the pair below 4)	`test_jsd_tie_pairs`
Treat L_{MM} like $V_\star$ (claim it ties)	`test_q_k_hat_terms`, `test_prefers`
Assume non-saturating has the same optimum ("tie")	`test_prefers`
Drop the $\frac{1}{2}$ in V_{LS} (report 2/3)	`test_ls_value_q2`
Finite $L_{\mathrm{NS}}(q_8)$ (ignore $D^* = 0$ off the data support)	`test_q_k_hat_terms`

3.3 (c) What does the problem test?

- **Topic:** GAN objectives and the divergences they minimize (minimax/JSD, non-saturating loss, LSGAN/Pearson-type χ^2), mode dropping vs. mode covering.
- **Problem type:** derivation, conceptual reasoning, and counterexample.
- **Core insight:** symmetric f-divergences such as JSD and the LSGAN criterion depend only on the support ratio, so they cannot distinguish dropping half the modes from spreading over twice the support; the map $k \mapsto 16/k$ produces exactly the ties. A solver with only surface knowledge repeats that "GANs penalize mode collapse" and treats the game value, the minimax generator term, and the non-saturating loss as interchangeable, when in fact they give three different preferences on the same pair.

3.4 (d) Testing AI systems

Each model ran as a coding agent in Cursor, given only `task/` in a fresh workspace and the same fixed prompt (no hints, no internet lookup of the problem, no access to tests or solutions). Each `answer.json` was graded with `tests/test_solution.py`.

Model (version)	Result	Tests passed
Claude Opus 5.5	Correct	8/8
GPT-5.6-sol	Correct	8/8
Grok 4.7	Correct	8/8

- **Claude Opus 5.5:** correct. It showed both criteria depend only on the support ratio and are monotone in it, which reduces ties to $jk = 16$. It also noted $V_\star = 2\,\mathrm{JSD} - \log 4$ and cross-checked by scanning $k \le 400$ with exact fractions.
- **GPT-5.6-sol:** correct. It gave the same reduction to $u = \min(k/4, 4/k)$ with an explicit derivative, and the cleanest written derivation of the three.
- **Grok 4.7:** correct, and the most thorough. It derived separate closed forms on each side of $k = 4$, proved the identity $\varphi(16/j) = \mathrm{JSD}(p, q_j)$ explicitly, and compared values exactly over $k \le 16$.

All three found the $\{1, 16\}$ pair, proved the tie lists complete, and separated $V_\star$, L_{MM}, and L_{NS} correctly. None fell into the anticipated traps. Current agentic models with code execution handle closed-form divergence calculations reliably, so this problem works best as a clean, verifiable check of the symmetric-divergence insight and of the difference between the three generator objectives, rather than as an item that breaks models.